

报表取数插件

目录

一、 概述.....	3
二、 插件基类.....	3
三、 注册插件.....	5
四、 插件事件.....	5
query 事件.....	6
getColumns 事件.....	6
setProgress 事件.....	7
getQueryParam 事件.....	7
getSelectedObj 事件.....	7
五、 案例演示.....	8
案例 1：从数据库中查询数据显示到报表，并冻结申请单号列.....	8
案例 2：增加金额统计行.....	12
六、 操作实践.....	13

一、概述

对于复杂的报表设计，通过配置数据源方式无法实现预期效果，此时需要使用到报表插件，

报表插件分为两类：

1、报表取数插件

2、报表表单插件

报表取数插件主要是针对报表列表设计，我们可以通过报表取数插件，完成复杂的业务逻辑

处理，从数据库中查询多张表中的数据显示到列表，也可以修改列表显示字段的效果，给报

表添加合计行等效果。

二、插件基类

报表取数插件基类 AbstractReportListDataPlugin

要实现报表取数插件，必须从插件基类 AbstractReportListDataPlugin 中派生

```

package kd.bos.entity.report;

import ...

538 inheritors

@SdkPlugin(
    name = "报表-查询取数插件"
)

public abstract class AbstractReportListDataPlugin {
    private IProgressRecorder progerssRecorder;
    private ReportQueryParam queryParam;
    private Object selectedObj;

    public AbstractReportListDataPlugin() {
    }
}

```

```

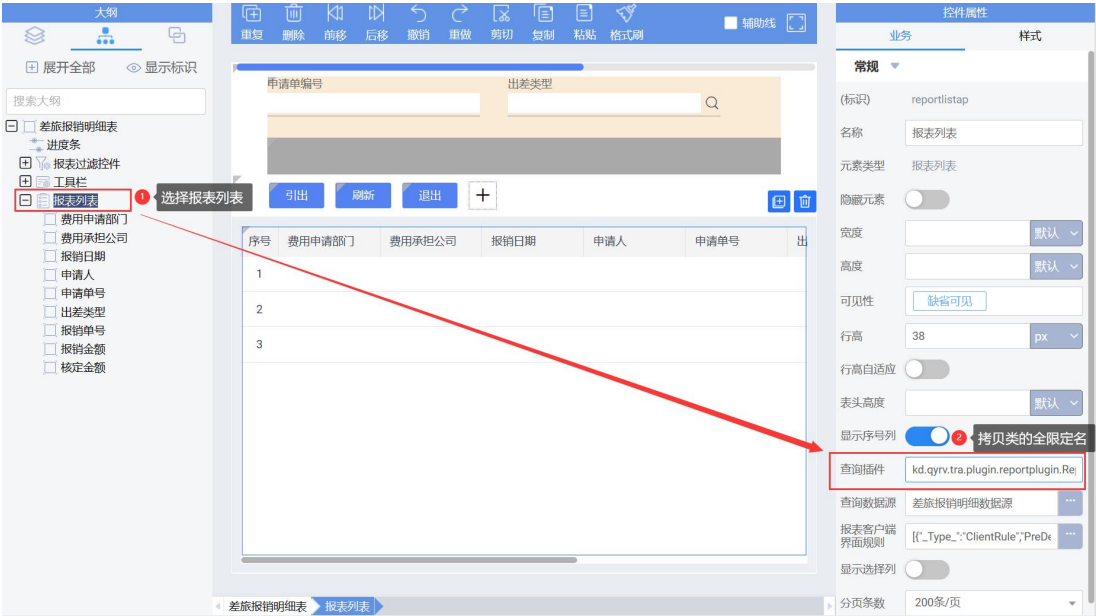
package kd.qyrv.tra.plugin.reportplugin;
import kd.bos.algo.DataSet;
import kd.bos.entity.report.AbstractReportColumn;
import kd.bos.entity.report.AbstractReportListDataPlugin;
import kd.bos.entity.report.ReportQueryParam;
import java.util.List;
public class ReportDemo1Plugin extends AbstractReportListDataPlugin {
    @Override
    public DataSet query(ReportQueryParam reportQueryParam, Object o) throws Throwable {
        return null;
    }
    @Override
    public List<AbstractReportColumn> getColumns(List<AbstractReportColumn> columns) throws Throwable {
        return super.getColumns(columns);
    }
}

```

繼承AbstractReportListDataPlugin

三、注册插件

选择“报表列表”，将插件类的全限定名拷贝到“查询插件”属性值中。



四、插件事件

AbstractReportListDataPlugin 内置了如下方法，供插件访问，用以获取操作执行上下文：

事件	说明
query	报表查询方法说明：参数 queryParam 为查询参数，selectedObj 为左 树（表）右表时选中左树（表）对象，左树时为节点 ID，左表时为选中行数据
getColumns	改变列信息 说明：参数 columns 为报表设计列，返回值为页面显示列

setProgress	设置进度，用于异步查询时界面显示进度
getQueryParam	获取查询条件
getSelectedObj	获取左表、左树选择行或节点

query 事件

查询数据，点击查询按钮会触发

事件参数

ReportQueryParam reportQueryParam 查询参数

Object o 左树（表）右表是选中左树（表）对象，左树时为节点 ID，左表时为选中行数据

@return DataSet 返回报表数据结果集

```
@Override
public DataSet query(ReportQueryParam reportQueryParam, Object o) throws Throwable {
    return null;
}
```

getColumns 事件

改变列信息 说明：参数 columns 为报表设计列，返回值为页面显示列

事件参数：

List<AbstractReportColumn> columns 报表设计列

@return List<AbstractReportColumn> 页面显示列

```
@Override
public List<AbstractReportColumn> getColumns(List<AbstractReportColumn> columns) throws Throwable {
    return super.getColumns(columns);
}
```

setProgress 事件

设置进度，用于异步查询时界面显示进度

```
public void setProgress(int percent) {
    this.progerssRecorder.setProgress(percent);
}
```

getQueryParam 事件

获取查询条件

```
public ReportQueryParam getQueryParam() {
    return this.queryParam;
}
```

getSelectedObj 事件

获取左表、左树选择行或节点

```
public Object getSelectedObj() {  
    return this.selectedObj;  
}
```

幻灯

五、案例演示

案例 1：从数据库中查询数据显示到报表，并冻结申请单号列

案例需求：自定查询，从数据库中查询数据显示到报表

- 1、统计差旅报销明细信息，结合“出差申请单”和“差旅报销单”
- 2、将申请单号列冻结

差旅报销明细表									
筛选条件									
展开过滤									
导出 刷新 退出									
#	申请单号	费用申请公司	费用承担部门	报销日期	申请人	出差类型	报销单号	申请金额	核定金额
1	TRIPREQ-20230614-00001001	环宇国际集团	环宇国际集团	2023-06-14		临时出差	CLBXD-20230614-00001001	¥1,500.00	¥1,500.00
2	TRIPREQ-20230614-00001002	环宇国际集团	环宇国际集团	2023-06-14		出长差	CLBXD-20230614-00001002	¥3,000.00	¥3,000.00

1.创建报表

2.界面设计

注意：在报表列表中增加以下字段，类型和单据中保持一致。

大纲

展开全部

显示标识

搜索大纲

差旅报销明细表

进度条

报表过滤控件

工具栏

报表列表

申请单号

费用申请公司

费用承担部门

报销日期

申请人

出差类型

报销单号

申请金额

核定金额

重复

删除

前移

后移

撤销

重做

剪切

复制

粘贴

格式刷

辅助线

申请单编号

出差类型

引出

刷新

退出

+

+

+

序号	申请单号	费用申请公司	费用承担部门	报销日期
1				
2				
3				

差旅报销明细表 报表列表

页面创建向导-PC端

公共模板

业务模板

通用查询报表模板

分组报表模板

轻分析报表模板

报表模板

模板图正在绘制中，请耐心等待

编码: bos_rpttpl

说明: 说明文案，模板内容说明

上一步

下一步

字段说明如下

序号	字段	标识	类型
1	申请单号	reqbillno	文本

2	费用申请公司	costcompany	组织
3	费用承担部门	costdepart	组织
4	报销日期	expdate	日期
5	申请人	user	用户
6	出差类型	triptype	基础资料
7	报销单号	bxbillno	文本
8	申请金额	hreqamount	金额
9	核定金额	hcheckamount	金额

编写插件

查询数据方法

```
@Override
public DataSet query(ReportQueryParam queryParam, Object o) throws Throwable {
    StringBuilder sbSql = new StringBuilder();
    //查询字段，最后留个空格
    sbSql.append("select T1.fbillno as gylv_reqbillno "); //申请单编号
    sbSql.append(",T1.fk_gylv_costcompany as gylv_costcompany "); //费用申请公司
    sbSql.append(",T1.fk_gylv_costdepart as gylv_costdepart "); //费用承担部门
    sbSql.append(",T1.fk_gylv_user as gylv_user "); //申请人
    sbSql.append(",T1.fk_gylv_triptype as gylv_triptype "); //出差类型
    sbSql.append(",T2.fbillno as gylv_bxbillno "); //报销单编号
    sbSql.append(",T2.fk_gylv_expdate as gylv_expdate "); //报销日期
    sbSql.append(",T2.fk_gylv_hreqamount as gylv_hreqamount "); //申请金额
    sbSql.append(",T2.fk_gylv_hcheckamount as gylv_hcheckamount "); //核定金额
    //出差申请单主实体表
    sbSql.append("from tk_gylv_tripreqbill T1 ");
    //差旅报销单主实体表
    sbSql.append("left join tk_gylv_tripreimbill T2 on T1.fbillno=T2.fk_gylv_srcbillno ");
    DataSet basedata = DB.queryDataSet(this.getClass().getName(), DBRoute.of( routeKey: "secd"), sbSql.toString());
    return basedata;
}
```

冻结列方法

```

@Override
public List<AbstractReportColumn> getColumns(List<AbstractReportColumn> columns) throws Throwable {
    for(int i = 0; i < columns.size(); i++) {
        ReportColumn rColumn = (ReportColumn) columns.get(i);
        String key = rColumn.getFieldKey();
        //冻结单据编号列
        if(key.equals("qyrv_reqbillno")) {
            rColumn.setFreeze(true);
            ColumnStyle style = new ColumnStyle();
            style.setBackColor("#AABBCC");
            style.setForeColor("#FFFFFF");
            rColumn.setStyle(style);
            columns.set(i, rColumn);
        }
    }
    return columns;
}

```

注册插件

注意：清空查询数据源

将报表取数插件全限定名拷贝到查询插件属性中

注意：如果试用了插件，要清除查询数据源的值

效果演示：将报表保存后，点击预览，点击查询。


```

public DataSet query(ReportQueryParam queryParam, Object o) throws Throwable {
    StringBuilder sbSql = new StringBuilder();
    //查询字段，最后留个空格
    sbSql.append("select T1.fbillno as gyrv_reqbillno "); //申请单编号
    sbSql.append(",T1.fk_gyrv_costcompany as gyrv_costcompany "); //费用申请公司
    sbSql.append(",T1.fk_gyrv_costdepart as gyrv_costdepart "); //费用承担部门
    sbSql.append(",T1.fk_gyrv_user as gyrv_user "); //申请人
    sbSql.append(",T1.fk_gyrv_triptype as gyrv_triptype "); //出差类型
    sbSql.append(",T2.fbillno as gyrv_bxbillno "); //报销单编号
    sbSql.append(",T2.fk_gyrv_expdate as gyrv_expdate "); //报销日期
    sbSql.append(",T2.fk_gyrv_hregamount as gyrv_hregamount "); //申请金额
    sbSql.append(",T2.fk_gyrv_hcheckamount as gyrv_hcheckamount "); //核定金额
    //出差申请单主实体表
    sbSql.append("from tk_gyrv_tripreqbill T1 ");
    //差旅报销单主实体表
    sbSql.append("left join tk_gyrv_tripreimbill T2 on T1.fbillno=T2.fk_gyrv_srcbillno ");
    DataSet basedata = DB.queryDataSet(this.getClass().getName(), DBRoute.of( routeKey: "secd"), sbSql.toString());

    //统计申请金额
    DataSet dataSet2=basedata.groupBy( strings: null).sum( s: "gyrv_hregamount").finish();
    dataSet2=dataset2.select( s: "'合计' as gyrv_reqbillno,null as gyrv_costcompany,null as gyrv_costdepart, " +
        "null as gyrv_user,null as gyrv_triptype,null as gyrv_bxbillno," +
        "null as gyrv_expdate,gyrv_hregamount,null as gyrv_hcheckamount");
    //需要两个dataset进行union操作，第一个dataset是报表数据，第二个dataSet2是自己计算的合计行数据
    return basedata.union(dataSet2);
}

```

实现步骤：在上面案例的 query 方法中，增加如图代码。

六、操作实践

报表实现

给办公用品管理系统配置一个报表，可查询当前登录人本年度领用物品的情况，领用数量有合计行。实现效果如下所示：

序号	物品	一月	二月		十二月
1	铅笔	5	5		2
2	笔记本	10	5		1
3	插座	2	6		4

合计		17	16		7
----	--	----	----	-------	---

提示

如果是统计行，可以使用 DataSet 的 union 方法进行结果集联合。

注意：在执行 SQL 语句时，别名的命名不要使用关键字。否则会报错。

```
select  name,sum(case when month(time)=1 then num end ) as one,
        sum(case when month(time)=2 then num end ) as two,
        sum(case when month(time)=3 then num end ) as three,
        sum(case when month(time)=4 then num end ) as four,
        sum(case when month(time)=5 then num end ) as five,
        sum(case when month(time)=6 then num end ) as six,
        sum(case when month(time)=7 then num end ) as seven,
        sum(case when month(time)=8 then num end ) as eight,
        sum(case when month(time)=9 then num end ) as nine,
        sum(case when month(time)=10 then num end ) as ten,
        sum(case when month(time)=11 then num end ) as eleven,
        sum(case when month(time)=12 then num end ) as twelve
from product
group by name;
```